

Terraform Interview Questions

DevOps Engineer (3–5 Years Experience) — Structured Answer Guide

1. How does an S3 backend with DynamoDB locking work?

- Terraform state (terraform.tfstate) is stored remotely in an S3 bucket instead of locally.
- S3 provides durable, versioned storage for the state file; bucket versioning allows rollback to a prior state if corruption occurs.
- A DynamoDB table is used purely for locking: before any plan/apply, Terraform writes a lock entry (keyed on the state file path/LockID) to the table.
- If a lock entry already exists, Terraform blocks the new operation and reports who holds the lock and since when.
- On completion (or failure with cleanup), Terraform deletes the lock item, releasing the table for the next operation.
- Note: as of Terraform 1.x with the S3 backend, native S3 conditional writes can also provide locking without DynamoDB, but DynamoDB locking is still the most common pattern in production.

2. Why is state locking important?

- Terraform state is the single source of truth mapping configuration to real infrastructure; concurrent writes corrupt it.
- Without locking, two engineers (or two CI pipeline runs) executing apply at the same time can both read the same state, make divergent changes, and the last write wins — silently discarding the other's changes.
- This can leave the state out of sync with actual infrastructure, causing drift, duplicate resource creation, or accidental deletion.
- Locking enforces serialized writes so only one apply/plan-with-write operation touches the state at a time.

3. What happens if the lock isn't released?

A stale lock typically results from a crashed process, killed CI job, network interruption, or manual termination mid-apply.

- Identify the lock holder/age from the error message (it includes Lock ID, who, and timestamp).
- Confirm no other operation is genuinely in progress (check CI status, ask the team).
- If confirmed stale, force-unlock with terraform force-unlock — this should be a deliberate, audited action, not routine.
- Investigate root cause (CI timeout, killed process) to prevent recurrence; consider adding pipeline timeouts and cleanup hooks.

4. How do you securely manage DB passwords and API keys?

- Never store secrets as plaintext values in .tf or .tfvars files committed to version control.
- Use a dedicated secrets manager: AWS Secrets Manager, AWS SSM Parameter Store (SecureString), Azure Key Vault, GCP Secret Manager, or HashiCorp Vault.

- Reference secrets via data sources at plan/apply time, e.g. data "aws_secretsmanager_secret_version", so the value is fetched at runtime rather than hardcoded.
- Mark sensitive variables with sensitive = true so Terraform redacts them from CLI output and plan diffs.
- Restrict state file access (encryption at rest via S3 SSE-KMS, bucket policies, IAM) since resolved secret values are written into the state in plaintext regardless of source.
- Use short-lived credentials (IAM roles, OIDC federation from CI) instead of long-lived static keys wherever possible.

5. Why should secrets never be hardcoded in .tf files?

- .tf files are typically committed to Git; once a secret is committed, it remains in version history even after later removal, requiring history rewriting to fully purge.
- Hardcoded secrets get exposed to anyone with repo read access, including CI logs, code review tools, and forks.
- They cannot be rotated without a code change and redeploy, increasing blast radius and operational friction.
- Static, hardcoded credentials violate least-privilege and audit requirements common in compliance frameworks (SOC 2, PCI-DSS, HIPAA).

6. How do you integrate Terraform with Vault or cloud secret managers?

- Vault: use the vault provider to read secrets at runtime via data "vault_generic_secret" or via Vault's dynamic secrets engines (e.g., AWS secrets engine issuing short-lived IAM credentials per apply).
- Vault Agent or the Vault Terraform provider can also inject secrets into environment variables consumed by the run.
- AWS: data "aws_secretsmanager_secret_version" or SSM data "aws_ssm_parameter" (with with_decryption = true) pull values directly into resource arguments.
- Azure: data "azurerm_key_vault_secret" referencing an existing Key Vault.
- GCP: data "google_secret_manager_secret_version".
- Common pattern: the secret manager is the source of truth; Terraform only reads references, and rotation happens independently of Terraform runs.

7. Explain create_before_destroy with a production example.

- By default, when a resource requires replacement, Terraform destroys the old resource first, then creates the new one — causing downtime.
- lifecycle { create_before_destroy = true } reverses this order: the new resource is created first, references are updated, and only then is the old resource destroyed.
- Production example: an aws_launch_template change that forces replacement (e.g., AMI ID update). With create_before_destroy, a new launch template is created and the Auto Scaling Group is pointed to it before the old template is removed, avoiding a window with no valid template.
- Another common case: aws_security_group referenced by other resources — replacing it outright would fail or break dependent resources if destroyed first; create_before_destroy creates the replacement and re-points dependencies before deleting the original.

- Caveat: requires unique naming (e.g., `name_prefix` instead of `name`) so the new and old resources can briefly coexist without a naming collision.

8. When would you use `prevent_destroy`?

- `lifecycle { prevent_destroy = true }` causes Terraform to error out and refuse any plan/apply that would destroy the resource — it does not prevent destruction outside Terraform.
- Use it for stateful, hard-to-recover resources: production databases (`aws_db_instance`), S3 buckets holding critical data, KMS keys, or core networking (VPCs) that downstream systems depend on.
- It acts as a guardrail against accidental destroy from a bad refactor, a forgotten resource rename (which Terraform treats as `destroy+create`), or a careless terraform destroy.
- It must be removed (or the resource explicitly excluded) when the resource genuinely needs to be decommissioned — it's a deliberate friction point, not a permanent lock.

9. How does Terraform resolve dependencies internally?

- Terraform parses all resource and data blocks and builds a Directed Acyclic Graph (DAG) representing dependencies between them.
- Implicit dependencies are inferred automatically from references (e.g., `aws_instance.web` using `aws_security_group.app.id` creates an edge in the graph).
- Explicit dependencies are declared via `depends_on` when there's no direct attribute reference but an ordering requirement still exists (e.g., IAM policy propagation).
- Terraform performs a topological sort on the DAG to determine execution order, and walks the graph executing operations on independent nodes in parallel (bounded by `-parallelism`, default 10).
- This graph-based model is also why terraform graph can visualize the dependency tree, and why a single resource error doesn't necessarily abort unrelated parallel operations.

10. How do you optimize Terraform for very large infrastructures?

- Split monolithic state into multiple smaller states aligned to logical boundaries (per environment, per service, per team) to reduce blast radius and plan/refresh time.
- Use remote state data sources (`terraform_remote_state`) or a tool like Terragrunt to compose outputs across states without duplicating resources.
- Adopt workspaces or directory-per-environment structures to avoid one giant state file representing every environment.
- Use targeted refresh/plan sparingly, and prefer `-refresh=false` in CI when state is known to be current, to cut down on API calls.
- Modularize repeated patterns into versioned modules to reduce config size and improve plan readability.
- Use a strict module registry/versioning strategy so changes are incremental rather than full-repo evaluations.

11. How do you reduce terraform plan execution time?

- Reduce the number of resources Terraform must refresh per run by splitting state (see above) — refresh time scales with resource count and provider API latency.

- Use -target sparingly for ad hoc, scoped plans during debugging (not as a standard workflow, since it can mask drift elsewhere).
- Increase -parallelism if the API rate limits of the provider allow more concurrent calls (default is 10).
- Cache provider plugins and modules (terraform init with a shared plugin cache directory: `TF_PLUGIN_CACHE_DIR`) to avoid re-downloading on every CI run.
- Avoid unnecessary data source lookups inside loops/modules that re-query the same external system repeatedly.
- For Terraform 1.5+, evaluate using the `refresh=false` flag combined with a scheduled separate drift-detection job, rather than refreshing on every plan.

12. How do you organize code for hundreds of resources?

- Break configuration into reusable modules (networking, compute, IAM, data) with clear input/output contracts rather than one large root module.
- Use a consistent directory layout, e.g., `environments/{dev,staging,prod}/` each calling shared modules from a `modules/` directory with environment-specific variables.
- Keep state per environment (or per logical domain) rather than a single shared state for everything.
- Standardize naming conventions and tagging via a shared locals or module so resources are consistently identifiable.
- Use variable validation blocks and tfvars per environment to avoid copy-pasted divergent configs.
- Document module interfaces (README per module) since at scale, configuration becomes a shared internal API across teams.

13. How do you integrate Terraform with Jenkins, GitHub Actions, or Azure DevOps?

- Pipeline stages typically follow: `checkout -> terraform fmt -check -> terraform init -> terraform validate -> terraform plan -> (manual approval) -> terraform apply`.
- Jenkins: use a declarative Jenkinsfile with stages calling the Terraform CLI, storing the plan as an artifact between stages, and credentials injected via Jenkins Credentials Binding (not hardcoded).
- GitHub Actions: use the `hashicorp/setup-terraform` action, run plan on `pull_request` events (commenting the plan output on the PR), and run apply only on merge to main, often gated by a required reviewers environment.
- Azure DevOps: similar pattern using the Terraform extension/task, with plan output published as a pipeline artifact and apply gated behind an Azure DevOps Environment approval check.
- In all cases, the remote backend (S3/DynamoDB, Azure Storage, GCS) ensures state consistency across distributed CI runners.

14. How do you implement approval before terraform apply?

- Generate a plan file (`terraform plan -out=tfplan`) and have the pipeline pause, posting the plan output for human review before any apply runs.
- GitHub Actions: use environment protection rules requiring designated reviewers before the apply job (which depends on the plan job) is allowed to run.

- Azure DevOps: use Environments with pre-deployment approvals tied to the apply stage.
- Jenkins: use the input step to pause the pipeline and require a manual click from an authorized user/group.
- Critically, apply should consume the exact saved plan file (terraform apply tfplan) rather than re-planning, so what was reviewed is exactly what gets applied — no drift between review and execution.

15. How do you handle rollbacks in Terraform pipelines?

- Terraform has no native 'rollback' command — rollback means applying a previous known-good configuration, not reverting state automatically.
- Practical approach: revert the infrastructure code to the last known-good commit and run a normal plan/apply through the pipeline.
- Maintain S3 state versioning (or equivalent) so a corrupted/bad state file can be restored to a prior version if needed.
- For destructive changes, prevent_destroy and create_before_destroy reduce the chance a rollback is even needed.
- Database and stateful resources generally need separate backup/restore strategies (snapshots) since Terraform rollback only affects the declared infrastructure, not data.
- Some teams use blue/green or canary patterns at the infrastructure level (e.g., new ASG/target group) so 'rollback' is a traffic shift rather than a Terraform operation at all.

16. count vs for_each — when would you use each?

- count creates N near-identical resource copies indexed numerically (resource[0], resource[1], ...).
- Problem with count: if an item is removed from the middle of a list, Terraform shifts indices, which can cause unrelated resources to be destroyed and recreated.
- for_each iterates over a map or set of strings, creating one instance per key, addressed by that key (resource["key_name"]) rather than a numeric index.
- Because for_each indexes by key, removing one item only affects that specific resource — others are untouched, making it far safer for resources that may grow/shrink over time.
- Use count for a fixed number of truly identical resources where order doesn't carry meaning (e.g., a fixed set of 3 NAT gateways one per AZ, defined by index).
- Use for_each when resources have distinguishing identity (different subnets, different IAM users, different S3 buckets) — which is the more common production case.

17. What is the difference between a resource and a data source?

- A resource block declares infrastructure Terraform should create, manage, update, and destroy — it owns the object's lifecycle.
- A data source block reads information about infrastructure that already exists (managed by Terraform elsewhere, or outside Terraform entirely) without managing its lifecycle.
- Resources appear in the state as managed objects subject to plan/apply/destroy; data sources are read-only lookups refreshed on each plan and are not destroyed by terraform destroy.
- Example: data "aws_vpc" "existing" looks up a VPC created manually or by another team, while resource "aws_subnet" "app" creates a new subnet inside that looked-up VPC.

18. How does Terraform ensure idempotency?

- Before every apply, Terraform refreshes its understanding of real infrastructure (via provider read APIs) and compares it against the desired state in configuration.
- It computes a diff between current state and desired configuration; if they already match, no API calls are made for that resource — running apply repeatedly with no config changes produces 'no changes' rather than re-creating resources.
- Providers are responsible for correctly implementing Create/Read/Update/Delete (CRUD) semantics against the target API so that re-running the same configuration converges to the same end state rather than duplicating resources.
- This diff-based, declarative model (rather than imperative scripting) is the core mechanism that makes repeated applies safe.

19. Why should provisioners be avoided in production?

- Provisioners (local-exec, remote-exec) step outside Terraform's declarative model, running imperative scripts that Terraform cannot track or diff — it has no insight into what the script actually changed.
- Failures are harder to recover from: a failed provisioner can leave a resource 'tainted' but doesn't roll back partial script side effects.
- They break idempotency guarantees, since arbitrary shell commands may not be safe to re-run.
- remote-exec requires network/SSH connectivity to the resource at apply time, creating tight coupling and fragility in pipelines (bastion access, security groups, key management).
- HashiCorp's own documentation treats provisioners as a last resort; preferred alternatives include cloud-init/user_data for instance bootstrapping, configuration management tools (Ansible, Chef, Puppet) run as a separate stage, or baking configuration into machine images (Packer) ahead of time.